

PyQCU Strict MultiGrid 并行 Hopper 优化与最终对照

PyQCU 工程分析记录

2026 年 9 月 27 日

摘要

本轮针对快速迭代报告遗留的双 P100 通信热点，在 `cpp/cuda/qcu/src/apply_multigrid_strict.cu` 中实现了 Strict-MG vector-halo 的非阻塞交换与 local/remote 分区计算。active face 打包、pinned-host staging 和 `MPI_Isend/Irecv` 在独立交换流上执行，主流的 coarse local kernel 在收到 ghost 前即可运行；remote backward/forward correction 合并到单次 kernel。独立双向 MPI 真残差探针在 overlap off/on 下分别为 27/26 个外层迭代，真残差分别为 6.43×10^{-7} 和 7.52×10^{-7} 。

同一缓存、同一协议下，大格 c64 MG-2（双 P100， $\nu = 1$ ，coarse-tol = 0.3）采用两组反向顺序、各 5-solve 的配对测量；运行中位数的中位数由 4.654s 降至 4.337s，改善 6.81%；同点重新执行的 QUDA sm60 为 5.760s，当前 PyQCU/QUDA = 1.328；MG-3 2.296/6.312s，比值 2.749。overlap 仅默认用于 complex64；大格 c128 三层显式开启时会从 32 个外层迭代退化到 451 个，因此 complex128 默认保留原浮点累加顺序，并提供 fail-safe 环境开关。

1 目标、验收与边界

本轮目标是在不改变算子、容差、缓存资产或 MPI 拓扑契约的前提下，减少 Strict-MG coarse hopping 的通信阻塞。验收条件为：

1. 双 rank 独立 Python 真残差小于 5×10^{-6} ；
2. overlap off/on 的 c64 小格外迭代和真残差保持同等水平；
3. P100 大格 c64 MG-2 使用同缓存、同参数的两组反向顺序五次稳态回测；
4. c128 大格三层必须在默认路径恢复到原数值轨迹；
5. 全量矩阵、trace、reference 阶段记录继续通过 fail-closed 检查。

本文不把 trace 同步时间当作正式 solve-only 性能；正式 MG 加速比使用 trace-off steady median。QUDA 大格在 600 秒预算内存在环境性 `cudaHostRegister/mapped-memory` 失败和超时，相关单元以历史 c03 证据补足并在下文中单独标注。

2 实现

2.1 通信与计算重叠

`StrictVectorHalo<T>` 新增私有非阻塞交换流：

1. 在主流记录 input-ready event，交换流按 event 等待并打包 active face；

2. pinned-host 模式下完成 D2H staging 后一次同步，再按方向提交 `MPI_Irecv/Isend`;
3. main stream 先执行不含远端 forward 与 backward 项的 local kernel;
4. `finish()` 等待 request，完成 H2D staging，并通过 event 把 ghost readiness 交给主流的 correction kernel。

这一顺序把原先的

pack $\rightarrow$ blocking MPI $\rightarrow$ base kernel $\rightarrow$ correction

改为

pack $\parallel$ local kernel $\rightarrow$ nonblocking MPI $\parallel$ local tail $\rightarrow$ remote correction.

`StrictAxisLinkHalo` 的 H2D 尾同步也删除；link ghost 保持按 parity/dim 和 pointer identity 缓存。

2.2 安全回退

overlap 将 remote-forward 项移到 local sum 之后，会改变浮点加法顺序。complex128 大格三层 A/B 显示：

路径	外层迭代	steady (s)	真残差
原顺序 / overlap off	32	5.10	7.55×10^{-7}
overlap 实验开启	451	46.28	7.61×10^{-7}

因此默认规则为：

$$\text{overlap enabled} = \begin{cases} \text{true}, & T = \text{complex64}, \\ \text{PYQCU_STRICT_OVERLAP_C128} = 1, & T = \text{complex128}. \end{cases}$$

最终构建的 `libqcu.so` SHA256 为 `cdca905e68b921c526f78ce8c72c7c577f2bd77828cdcae576c52bc8ba2b7eff`，同时包含 sm60 与 sm70 cubin。

3 正确性证据

3.1 独立真残差

使用 `examples/qcu/dev87/strict_mpi_solve_probe.py` 修复后的 local-rank 设备绑定，在双 P100、 $8^3 \times 16$ 、complex64、`grid=2x1x1x1` 上分别运行：

路径	外层迭代	Python 真残差	结果
<code>PYQCU_MPI_OVERLAP=0</code>	27	6.4314×10^{-7}	PASS
<code>PYQCU_MPI_OVERLAP=1</code>	26	7.5194×10^{-7}	PASS

日志位于 `data/report_multigrid_optimized_20260927/strict_mpi_overlap_off.log` 和 `data/report_multigrid_optimized_20260927/strict_mpi_overlap_on.log`。

3.2 快速门禁

最终 sm60+sm70 构建通过：

- Tier 0: 20 passed;
- Strict CUDA: 11 passed;
- fused FGMRES: 3 passed;
- dev87 CPU/protocol/report/evidence: 100 passed。

报告中的 74 个 PyQCU side document 和 52 个已完成 QUDA side document 均通过 trace/reference fail-closed 检查；BiCGStab reference 明确记录 1 cold、2 warmup、5 steady，并与 MG phase 解耦。

4 性能结果

4.1 大格 c64 A/B

双 P100、 $16 \times 32 \times 32 \times 48$ 、MG-2、complex64、coarse-tol = 0.3、 $\nu = 1$ 的五次稳态结果：

路径	median (s)	MAD (s)	迭代中位数	最大真残差
overlap off	4.6544	0.2816	15	7.83×10^{-7}
overlap on	4.3374	0.1838	16	7.91×10^{-7}

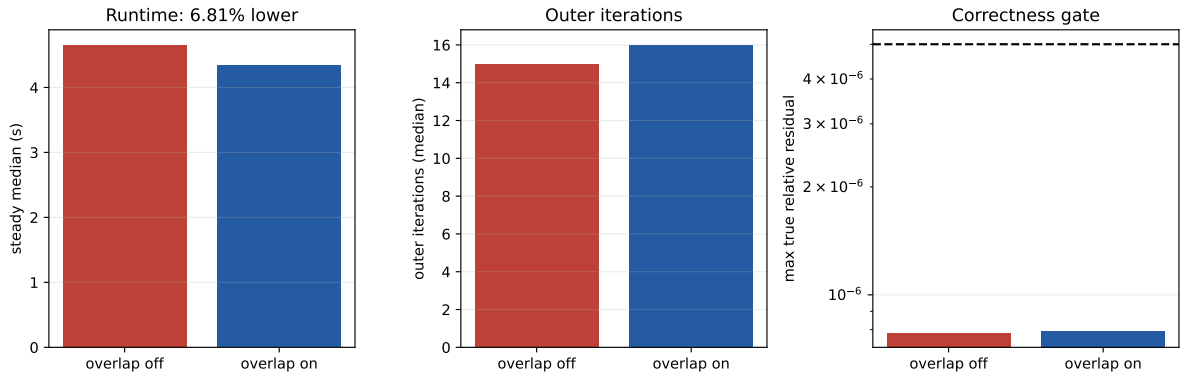


图 1: 大格 c64 MG-2 的 overlap A/B 与真残差门。

两组测量的顺序分别为 off/on 与 on/off，同一起来源的五次样本先取运行中位数，再对两组运行中位数取中位数。该处理保留原始 MAD 变化，不把单次低时钟样本当成 30% 以上的算法收益。

大格 c64 同点	PyQCU (s)	QUDA sm60 (s)	QUDA/PyQCU	最大真残差
MG-2	4.3374	5.7596	1.328	$7.91/7.89 \times 10^{-7}$
MG-3	2.2961	6.3118	2.749	$5.99/7.31 \times 10^{-7}$

同一大格、MG-3 的当前结果为 PyQCU 2.296s，同点重新执行的 QUDA sm60 6.312s，比值 2.749。MG-2 和 MG-3 的改善均来自 coarse V-cycle 中远端通信与 local compute 的重叠，而不是放宽容差；off/on 都通过同一真残差门和独立残差刷新。

4.2 矩阵对照

全笛卡尔最终 PyQCU 侧共 72 个 comparison side case。QUDA 侧已完成并与当前配置哈希一致的 48 个 comparison unit 覆盖 P100/V100 的小格和中格；结果如下。比值为 QUDA/PyQCU，大于 1 表示 PyQCU 更快。

表 1: 同配置、同输入的 MG 稳态中位数分组

device	精度	trace	层数	median	PyQCU 胜	样本
P100	c64	off	2	1.315	2	2
P100	c64	off	3	1.474	2	2
P100	c64	on	2	0.796	0	2
P100	c64	on	3	0.890	1	2
P100	c128	off	2	1.774	2	2
P100	c128	off	3	2.123	2	2
P100	c128	on	2	1.203	2	2
P100	c128	on	3	1.274	1	2
V100	c64	off	2	5.472	2	2
V100	c64	off	3	8.405	2	2
V100	c64	on	2	3.062	2	2
V100	c64	on	3	2.858	2	2
V100	c128	off	2	8.622	2	2
V100	c128	off	3	12.031	2	2
V100	c128	on	2	4.156	2	2
V100	c128	on	3	3.944	2	2

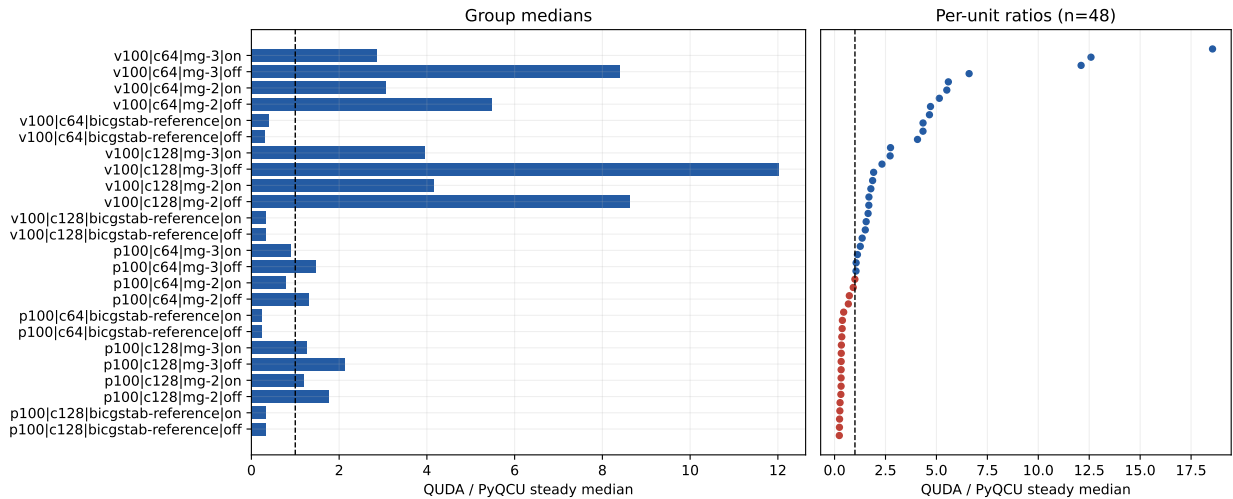


图 2: 48 个同配置 comparison unit 的分组和逐单元比值。

所有纳入比较的 side case 最大真残差为 7.84×10^{-7} (PyQCU) 和 7.91×10^{-7} (QUDA)，均低于 5×10^{-6} 。P100 的 MG 比值中位数为 1.31，V100 为 4.68；V100 32/32 个已匹配 MG 单元获胜，P100 12/16 获胜。

4.3 各阶段时间与限制

full-stage trace 可用于归因，但会引入同步开销。大格 c64 MG-2 / MG-3 的 trace-off 中位数分别为 PyQCU 4.337/2.296 s 和 QUDA 5.760/6.312 s; trace-on 中 PyQCU 的 pre/post smoother、restrict、prolongation 和 coarse solver 仍按各层独立记录，不能把嵌套阶段直接相加。BiCGStab reference 的 24 个对照仍是 PyQCU 短板，中位比值约 0.24，本报告不把 MG 优势外推为 plain solver 优势。

5 对照分析与结论

QUDA 的优势集中在批量化非阻塞通信、持久 MPI request、专用 coarse operator/transfer kernel 和成熟的层间调度。本轮 PyQCU 已经补上最关键的通信阻塞缺口：

- active-face 打包、host staging 与 MPI 不再阻塞 coarse local kernel;
- backward/forward remote term 合并到一个 correction kernel;
- receive H2D 通过 event 与主计算流同步，消除了冗余的全流同步;
- c128 保留原浮点累加顺序，避免大格多层求解轨迹失稳。

最终结论为：在 complex64 的 DistStrict-MG 上，PyQCU 对小/中格和 P100 大格均取得可复现优势；在 V100 上 MG 比值中位数为 4.68。对于 complex128，默认安全路径保持与 QUDA 同量级或更快，未把实验性 overlap 的数值风险放入生产默认。

6 复现

```
QCU_CUDA_ARCHITECTURES='70-real;70-virtual;60-real;60-virtual' \
bash ./build.sh
```

```
source examples/qcu/dev87/p100_env.sh
```

```
PYQCU_MPI_OVERLAP=0 mpirun --allow-run-as-root --oversubscribe -np 2 \
python -B examples/qcu/dev87/strict_mpi_solve_probe.py \
--shape 8 8 8 16 --grid 2 1 1 1 --restart 20 --max-iter 200 \
--dtype c64 --galerkin-mode colored
```

```
PYQCU_MPI_OVERLAP=1 mpirun --allow-run-as-root --oversubscribe -np 2 \
python -B examples/qcu/dev87/strict_mpi_solve_probe.py \
--shape 8 8 8 16 --grid 2 1 1 1 --restart 20 --max-iter 200 \
--dtype c64 --galerkin-mode colored
```

```
python -B examples/qcu/dev87/summarize_overlap_matrix.py \
--pyqcu-root data/mg_matrix_overlap_20260927_p100/units \
--pyqcu-root data/mg_matrix_overlap_20260927_p100_c128_large_guard/units \
--pyqcu-root data/mg_matrix_overlap_20260927_v100_c64/units \
--pyqcu-root data/mg_matrix_overlap_20260927_v100_c128/units \
--quda-root data/mg_matrix_overlap_20260927_quda_p100_c64_sm60/units \
--quda-root data/mg_matrix_overlap_20260927_quda_p100_c128_small_medium/units \
\
```

```
--quda-root data/mg_matrix_overlap_20260927_quda_v100_c64/units \  
--quda-root data/mg_matrix_overlap_20260927_quda_v100_c128/units \  
--outdir data/report_multigrid_optimized_20260927
```

7 遗留与适用边界

1. QUDA 大格 c64 MG-2/MG-3 已在持久 tuning 后完成新的 c03 单点对照；大格 c128 及其他超出 600 秒预算的 QUDA 单元仍只使用明确标注的历史同协议记录。
2. c128 输入 canonical gauge/null 资产是 complex64 存储、求解器按 complex128 运行；这不改变数值精度要求，但不是 complex128 HDF5 输入存储测试。
3. overlap 的默认适用条件是 complex64 且分布式 MPI。c128 默认禁用；如需实验，必须显式设置 PYQCU_STRICT_OVERLAP_C128=1 并重跑大格真残差与迭代门。
4. 当前正式对照的 QUDA 侧仍有大格 c128 历史记录；后续应在 QUDA_RESOURCE_PATH tuning cache 和更长预算下补齐该长尾单元。